

DOCUMENTATION FONCTIONNELLE

Système de prédiction de la consommation énergétique

Projet MLOps DPE — Stage 2026

Projet	MLOps DPE
Objectif	Prédire la consommation énergétique d'un logement (kWh/an)
Utilisateur final	Consommateur souhaitant estimer sa consommation (type app EDF)
Technologies	Python, scikit-learn, MLflow, DVC, FastAPI, Docker

1. Vision du projet

Ce projet a pour objectif de construire un système complet qui prédit la consommation énergétique annuelle (kWh/an) d'un logement à partir de ses caractéristiques, et de l'industrialiser avec des outils MLOps pour qu'il soit utilisable en production.

Concrètement, l'idée finale est la même que les applications mobiles proposées par EDF ou Engie : un consommateur saisit des informations sur son logement (surface, année de construction, type de chauffage...) et l'application lui retourne une estimation de sa consommation énergétique annuelle.

MLOps = Machine Learning + DevOps. C'est l'art de faire passer un modèle de la phase de recherche à la phase de production de façon fiable et reproductible.

2. Pipeline complet du projet

Le projet suit un enchaînement logique et linéaire, de la donnée brute jusqu'à l'application accessible en ligne :

Données (DVC) → Entraînement (DPE.py)
Entraînement (DPE.py) → Suivi des expériences (MLflow)
Suivi des expériences (MLflow) → Meilleur modèle (.pkl)
Meilleur modèle (.pkl) → API de prédiction (FastAPI)
API de prédiction (FastAPI) → Déploiement (Docker en locale)
Déploiement (Docker) → Application accessible sur inter

3. Description

3.1 DPE.py — Le cœur du projet

C'est le script principal d'entraînement. Il réalise 4 actions dans l'ordre :

1. Charge les données depuis data/DPE.csv
2. Entraîne 5 modèles différents : Ridge, Lasso, ElasticNet, RandomForest, GradientBoosting
3. Compare leurs performances avec 3 métriques : R^2 , MAE, MAPE
4. Sauvegarde automatiquement le meilleur modèle en local (.pkl) et dans MLflow

Le meilleur modèle est sélectionné selon le R^2 sur le jeu de test. Plus le R^2 est proche de 1, meilleure est la prédiction.

3.2 data/DPE.csv — Les données sources

Le dataset contient des diagnostics de performance énergétique de logements français. Chaque ligne correspond à un logement avec ses caractéristiques (surface, année de construction, type de chauffage, etc.) et sa consommation réelle en kWh/an.

3.3 DVC — Versionnement des données

DVC (Data Version Control) fonctionne comme Git, mais pour les données. Il surveille le fichier DPE.csv et garantit que si les données changent, le pipeline peut être relancé automatiquement pour produire un nouveau modèle.

- data/DPE.csv.dvc → fichier de surveillance du dataset
- dvc.yaml → recette du pipeline ("lance DPE.py avec ces données → produit ce modèle")
- dvc/ → dossier interne de DVC, à ne pas modifier

3.3.1 DVC — DVC et évolution des données (cas réel)

Dans ce projet, DVC (Data Version Control) est utilisé pour assurer le suivi et la gestion des données, de manière similaire à Git pour le code.

Le fichier DPE.csv est versionné via DVC, ce qui permet de détecter automatiquement toute modification du dataset.

Concrètement, lorsqu'une mise à jour des données est effectuée (par exemple ajout de nouvelles lignes ou nouvelles catégories), il est nécessaire de versionner cette nouvelle version du dataset avec les commandes suivantes :

```
dvc add data/DPE.csv ou git add data/DPE.csv.dvc
git commit -m « mise à jour du dataset »
```

Pour pallier ce problème, DVC permet de relancer automatiquement le pipeline d'entraînement défini dans le fichier dvc.yaml, en utilisant la nouvelle version du dataset.

Cette exécution se fait via la commande :

```
dvc repro
```

Ce pipeline exécute les étapes de prétraitement et de réentraînement afin de générer un modèle mis à jour, plus représentatif des données actuelles.

Une fois le nouveau modèle entraîné, celui-ci peut être enregistré et versionné dans MLflow, puis promu en production afin d'être utilisé par l'API.

Cela garantit que l'API utilise toujours la version la plus récente et la plus pertinente du modèle. Ce mécanisme assure une cohérence entre les données, le modèle et les prédictions, tout en facilitant l'automatisation, la traçabilité et la reproductibilité du système dans un contexte MLOps.

3.4 MLflow — Suivi des expériences et gestion du modèle en production

MLflow est le tableau de bord central du projet. Il remplit deux rôles essentiels :

Rôle 1 — Suivi des expériences :

À chaque entraînement, MLflow enregistre automatiquement les paramètres de chaque modèle testé et leurs métriques de performance. Cela permet de comparer visuellement tous les runs et de retrouver le meilleur.

- mlflow.db → base de données SQLite qui stocke toutes les expériences
- mlartifacts/ → dossiers 0, 1, 2, 3 correspondant aux 4 expériences lancées
-

On a accès a toutes les informations sur l'interface

Run Name	Created	Duration	Source	Model	CV_R2_mean	CV_R2_std	TOST_MAE	TOST_R2	Parameters
RandomForest	1 month ago	17.2s	DPE.py	new_model_v0.0	0.98077427...	0.00133006...	1001.432448...	0.96707536...	GradientDescent
RandomForest	1 month ago	17.5s	DPE.py	new_model_v0.1	0.981561985...	0.001566716...	893.389276...	0.96707536...	RandomForest
DecisionTree	1 month ago	8.5s	DPE.py	model	0.22292735...	0.000052162...	7609.374384...	0.00001177...	DecisionTree
Linear	1 month ago	8.5s	DPE.py	model	-0.00162112...	0.001738938...	8405.780326...	-0.00449297...	Linear
Ridge	1 month ago	21.5s	DPE.py	model	0.888987373...	0.000602950...	3131.053033...	0.075019678...	Ridge
GradientDescent	1 month ago	6.1s	DPE.py	model	0.98077427...	0.00133006...	1001.432448...	0.96707536...	GradientDescent
RandomForest	1 month ago	6.6s	DPE.py	model	0.981561985...	0.001566716...	893.389276...	0.96707536...	RandomForest
DecisionTree	1 month ago	5.0s	DPE.py	model	0.22292735...	0.000052162...	7609.374384...	0.00001177...	DecisionTree
Linear	1 month ago	6.4s	DPE.py	model	-0.00162112...	0.001738938...	8405.780326...	-0.00449297...	Linear
Ridge	1 month ago	10.0s	DPE.py	model	0.888987373...	0.000602950...	3131.053033...	0.075019678...	Ridge

Figure 1: Interface MLflow

Rôle 2 — Registre de modèles (Model Registry) :

MLflow permet de promouvoir un modèle en "production". Quand on tague un modèle avec l'alias @production dans l'interface MLflow, c'est ce modèle précis que l'API va charger et utiliser pour répondre aux utilisateurs. C'est ce mécanisme qui permet de changer de modèle en production sans toucher au code de l'API.

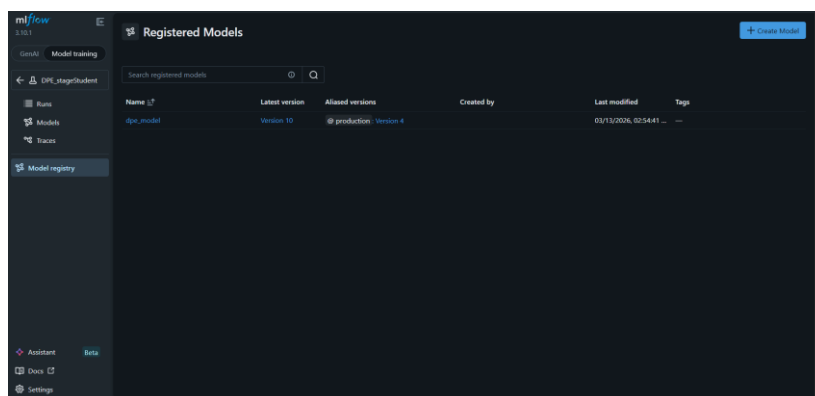


Figure 2 : Interface MLflow (Mise en production)

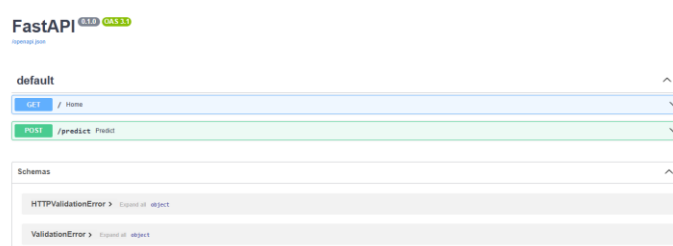
C'est exactement ce que font les grandes plateformes : le modèle en production peut être mis à jour sans couper le service.

3.5 api.py — L'API de prédiction (FastAPI)

C'est l'interface entre le modèle et l'utilisateur final. L'API expose deux points d'accès :

Endpoint	Méthode	Description
/	GET	Page d'accueil — affiche un exemple de données d'entrée
/predict	POST	Reçoit les caractéristiques d'un logement et retourne la consommation prédite en kWh/an

L'API charge le modèle directement depuis le registre MLflow via l'alias `@production`. Ainsi, si on change le modèle en production dans MLflow, l'API utilisera automatiquement le nouveau modèle au prochain démarrage.

Figure 3 : <http://localhost:8001/docs#>

Dans ce projet, l'API est exposée via l'interface Swagger générée automatiquement par FastAPI (accessible via /docs). Cette interface permet de tester facilement les endpoints, comme /predict, en envoyant des requêtes directement depuis le navigateur.

Cependant, cette représentation reste volontairement simple et technique. Aucune interface utilisateur (frontend) n'a été développée à ce stade, car l'objectif est de connecter directement l'API à une application externe, comme une application mobile ou un service web.

Ainsi, l'API joue le rôle de backend de prédiction : elle reçoit des données (par exemple depuis une application mobile), exécute le modèle de machine learning, puis renvoie une prédiction. L'interface Swagger sert donc uniquement d'outil de test et de validation durant le développement, et non comme interface finale destinée aux utilisateurs.

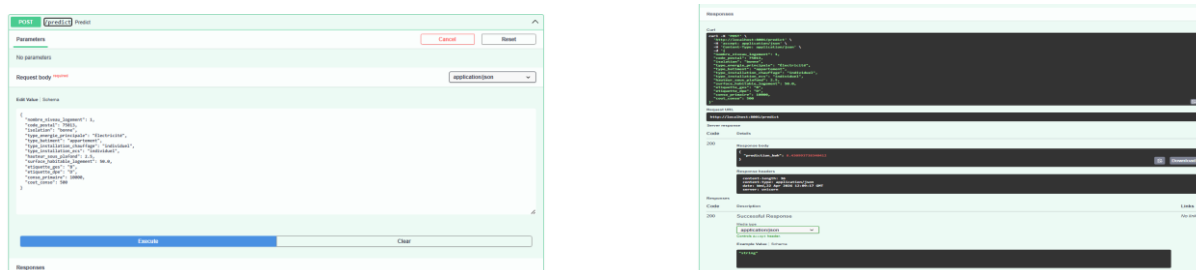


Figure 4 : Exemple de prédiction

3.6 Docker — Conteneurisation et déploiement

Docker permet d'empaqueter l'API et toutes ses dépendances dans une "boîte" autonome appelée conteneur. Cette boîte peut être déployée sur n'importe quel serveur dans le monde sans configuration supplémentaire.

- Dockerfile → la recette de construction de l'image Docker
- requirements.txt → liste des bibliothèques Python incluses dans l'image

C'est exactement le principe des applications mobiles EDF/Engie : le serveur qui tourne derrière l'app est un conteneur Docker déployé dans le cloud.

4. Comment lancer le projet

Il est obligatoire d'ouvrir deux terminaux en parallèle, car les deux services doivent fonctionner simultanément, et il faut se placer dans le répertoire où se trouve le code source du projet avant d'exécuter les commandes.

Terminal 1 — Lancer MLflow

MLflow doit toujours être lancé en premier, avant Docker.

```
set MLFLOW_SERVER_ALLOWED_HOSTS=*
mlflow server --host 0.0.0.0 --port 5000
```

MLflow est ensuite accessible dans le navigateur à l'adresse :

```
http://127.0.0.1:5000
```

Terminal 2 — Lancer l'API via Docker

Une fois MLflow lancé, démarrer le conteneur Docker dans un second terminal :

```
docker run -p 8001:8000 -e MLFLOW_TRACKING_URI=http://host.docker.internal:5000 dpe-api
```

Si l'image Docker n'a pas encore été créée, il faut d'abord la construire à partir du projet :

```
Docker build -t dpe-api
```

Cette commande permet de créer une image Docker nommée `dpe-api`, contenant l'ensemble de l'application (API, modèle, dépendances).

L'API est ensuite accessible à :

```
http://127.0.0.1:8001/docs
```

L'interface `/docs` est générée automatiquement par FastAPI. Elle permet de tester l'API directement depuis le navigateur, sans outil externe.

5. Workflow de mise en production d'un modèle

Voici les étapes pour mettre à jour le modèle en production :

1. Lancer `DPE.py` pour entraîner de nouveaux modèles → ils apparaissent dans MLflow
2. Ouvrir l'interface MLflow (`http://127.0.0.1:5000`) et comparer les runs
3. Sélectionner le meilleur modèle et lui attribuer l'alias `@production` dans le registre
4. Redémarrer le conteneur Docker → l'API chargera automatiquement le nouveau modèle

Ce workflow reproduit exactement ce qui se fait en entreprise : les Data Scientists entraînent, comparent et valident un modèle, puis le mettent en production sans toucher au code de l'API.

6. Vision finale — Vers une application grand public

La version locale (sur ta machine) est la première étape. La vision finale est de déployer ce système sur un serveur distant pour qu'il soit accessible à tous via internet, exactement comme les applications mobiles proposées par EDF ou Engie.

Étape	Description
Étape 1 — Local	Le système tourne sur ta machine. MLflow + Docker + API fonctionne ensemble.
Étape 2 — Serveur	Le conteneur Docker est déployé sur un serveur cloud (AWS, Azure, GCP...). L'API devient accessible depuis n'importe quelle adresse IP.
Étape 3 — Application	Une interface web ou mobile appelle l'API <code>/predict</code> . L'utilisateur final saisit ses données et reçoit sa prédiction en temps réel.

Ce projet démontre qu'avec les bons outils MLOps, un modèle de Machine Learning peut passer du stade de prototype à celui d'application utilisable par des millions de personnes, de façon fiable et maintenable.

Le MLOps permet d'organiser tout le cycle de vie d'un modèle de machine learning, depuis sa création jusqu'à son utilisation en production.

Dans un premier temps, le modèle est entraîné à partir des données disponibles. Une fois entraîné, il est évalué et testé afin de vérifier ses performances et sa fiabilité. Seul le meilleur modèle est ensuite sélectionné et enregistré (par exemple avec MLflow) afin de garantir une gestion des versions et une traçabilité.

Ensuite, le modèle est intégré dans une API (comme FastAPI) afin de pouvoir être utilisé pour faire des prédictions. Cette API est testée localement, généralement à l'aide de Docker, pour vérifier que tout fonctionne correctement dans un environnement contrôlé.

Une fois validé en local, le système peut être déployé sur un serveur (cloud ou infrastructure dédiée), afin de rendre le modèle accessible à des utilisateurs réels, via une application web ou mobile.